Jyro Language Syntax

Jyro2025 specification. This document produced September 2025.

Table of Contents

- About Jyro
- Introduction to the Language
- Comments
- Variables and Assignment
  - Variable Declaration (`var`)
  - Assignment
- Expressions
  - Expression Statement
  - Expression Hierarchy
    - Ternary Conditional Operator (`?:`)
    - Logical OR (`or`)
    - Logical AND (`and`)
    - Equality (`==` , `!=`)
    - Relational (`<` , `<=` , `>` , `>=` , `is`)
    - Additive (`+` , `-`)
    - Multiplicative (`*` , `/` , `%`)
    - Unary (`not` , `-`)
  - Primary Expressions
- Literals and Data Structures
  - Basic Literals
  - Object Literals
  - Array Literals
- Control Flow
  - Conditional Execution (`if` , `then` , `else` , `end`)
  - Multi-way Branching (`switch` , `case` , `default` , `end`)
- Iteration and Looping
  - Conditional Iteration (`while` , `do` , `end`)
  - Collection Iteration (`foreach` , `in` , `do` , `end`)
  - Loop Control (`break` , `continue`)
- Function Integration
  - Function Calls
- Script Termination
  - Return Statement (`return`)
- Type Operations
  - Type Checking (`is`)
- Lexical Conventions
  - Identifiers
  - Number Literals
  - String Literals
- Reserved Keywords
- Operator Precedence Summary

About Jyro

Jyro is an imperative, dynamically-typed interpreted scripting language built for heavy data transformation and processing workflows. Scripts operate on a **mutable data context** supplied by the host, eliminating the complexity of object creation, memory management, and return value handling while supporting full transformation capabilities via **familiar procedural programming constructs**. This architecture makes Jyro ideal for **embedded scripting** scenarios ranging from trusted automation tasks to **untrusted user-generated scripts** in multi-tenant applications, where the **fully isolated execution model** and explicit **host-function allowlisting** prevent privilege escalation and system compromise without sacrificing functionality.

Introduction to the Language

The language emphasizes explicit syntax with structured block delimiters and provides runtime type checking capabilities while maintaining simplicity and readability.

The language operates on a single data object context, allowing scripts to query, modify, and transform data through a controlled execution environment. Jyro follows a procedural programming paradigm where statements execute sequentially, with support for conditional branching, iterative constructs, and host-provided function integration.

Type annotations in Jyro are optional but supported, serving primarily as documentation and potential optimization hints rather than compile-time constraints. The runtime performs type checking through the `is` operator, enabling scripts to make decisions based on the actual types of values encountered during execution. This approach provides flexibility while maintaining type awareness when needed.

Block structures in Jyro require explicit termination using the `end` keyword, creating clear visual boundaries for control flow constructs. This design choice eliminates ambiguity in nested structures and provides consistent syntax patterns across all block-based statements including conditionals, loops, and switch statements.

Object and array construction requires explicit type keywords (`object` and `array` respectively) rather than relying on bracket notation alone. This explicit syntax improves readability and removes potential parsing ambiguities, making the code's intent clear at the lexical level.

Logical operations use word-based operators (`and` , `or` , `not`) rather than symbolic representations, aligning with the language's emphasis on readability and explicit intent. The operator precedence follows standard mathematical conventions while maintaining clear distinctions between logical and arithmetic operations.

Function capabilities are limited to calling host-provided functions rather than defining custom functions within the script. This constraint keeps the language focused on data transformation tasks while allowing extensibility through the host environment. The runtime does not support implicit type conversions, requiring explicit handling of type differences in expressions and operations.

Whitespace is insignificant for all language constructs except for comments, which use a newline to terminate. Statement blocks are deliminated explicitly, and whitespace is therefore ignored. The Jyro convention is to indent each block level using either 4 spaces or a tab. The examples shown in this document reflect that convention.

The standard file extension for Jyro scripts is `.jyro` . The content type can be specified as `Content-Type: application/vnd.mesch.jyro`

Jyro is a case sensitive language. The case conventions are:

- **keywords**: lowercase (e.g. `foreach`)
- **local variables**: lowerCamelCase (e.g. `var myFoo = 42`)
- **iterator variables**: Follow the array key (e.g. `foreach item in Data.items` or `foreach Item in Data.Items`)
- **host function calls**: PascalCase (e.g. `Today()`)

- **script names**: PascalCase (e.g. `CallScript("ProcessVipOrders.jyro", vipOrders)`)
- **the `Data` object**: PascalCase (i.e. upper-case `D` , lower-case everthing else: `Data`)
- **object property names**: These are typically string literals in Jyro (e.g. `object { "propertyName": value }`), so the convention depends on the data domain.
- **added properties**: Again, follow the data domain.
- **interpolated keys**: Depends on the expression that generated the key.

A Jyro program consists of a sequence of statements that execute in order from top to bottom. Each statement represents a complete instruction such as variable declaration, assignment, expression evaluation, or control flow operation.

```
Program          = { Statement } ;
Comment          = "#" { Character } EndOfLine ;
```

At the heart of every Jyro script execution is the `Data` context, a special built-in identifier that represents the primary data object being processed. The `Data` context is provided by the host environment when a script begins execution and serves as both the input to the transformation process and the output that will be returned to the host upon completion.

`Data` is automatically available in every script without declaration and maintains its identity throughout the entire execution lifecycle. The host environment populates `Data` with the initial data structure before script execution begins, and any modifications made to `Data` during script execution are reflected in the final result returned to the host. This design creates a clear contract between the host and script: the host provides data for transformation, and the script modifies that data in place. Hosts may pass complex objects of any practical size to Jyro, or may simply pass a null object `{}` if the target script is intended to construct a data structure from scratch.

Scripts may not declare a new variable named `Data` using `var Data =` syntax, as this would shadow the built-in context and break the host-script communication contract. Similarly, direct assignment to the `Data` root itself is prohibited - scripts cannot execute statements like `Data = newObject` as this would replace the entire context rather than transforming it.

However, the language provides full access to modify the contents and structure of the `Data` context through property assignment, indexing, and method calls. Scripts can freely execute operations such as `Data.newProperty = "value"` , `Data["dynamicKey"] = calculation` , or `Data.existingArray[index] = updatedValue` . This approach ensures that while the `Data` context remains stable and accessible throughout execution, its contents can be completely transformed to meet the script's processing requirements.

> ⚠️ The script has complete control over the `Data` object that was passed to it, therefore it is the host's responsibility to ensure that `Data` only contains objects that are safe for reading and writing. This is especially important in untrusted scenarios, where the host does not trust the script that is being run (e.g. multi-tenanted applications with scripting support).

> ⚠️ Never directly evaluate an expression passed back on the `Data` object. Doing so could lead to privilege escalation and host compromise. Hosts should treat all data returned from scripts as potentially malicious content that should be sanitized or validated before any further processing.

When script execution completes, either by reaching the end of the statement list or encountering a `return` statement, the current state of the `Data` context is returned to the host environment. This return mechanism ensures that all modifications made during script execution are captured and made available to the calling code, completing the data transformation workflow.

Comments

Comments begin with `#` and continue to the end of the line. Comments may be placed on their own line or at the end of a statement. Multi-line comments are written with a `#` at the beginning of each line. Comments are typically stripped out of a Jyro script when it is compiled and have no effect on execution.

Valid Syntax:

```
# This is a single-line comment
var counter = 0

# This is a
# multi-line comment
ProcessData()

# Comments can be split...
DoFoo()  #... with statements in between
```

Variables and Assignment

Variable Declaration (`var`)

Variable declarations introduce new identifiers into the current scope with optional type annotations and initialization values. The `var` keyword explicitly marks the creation of a new variable, distinguishing declaration from assignment operations.

```
VariableDecl     = "var" Identifier [ ":" Type ] [ "=" Expression ] ;
Type             = "number" | "string" | "boolean" | "object" | "array" ;
```

Type annotations provide documentation of expected value types and may be used by development tools for analysis, though the runtime does not enforce these constraints. Variables declared without initial values are set to `null` by default.

Valid Syntax:

```
var counter
var name: string
var total: number = 100
var isActive: boolean = true
var data = GetSomeData()
```

Invalid Syntax:

```
var                    # Missing identifier
var 123invalid         # Identifier cannot start with digit
var name: unknown      # Unsupported type
var x, y               # Multiple declarations not supported
```

Variable Scope and Blocks

Variables are scoped to the block in which they are declared, following lexical scoping rules where inner blocks have access to variables declared in outer blocks. Each control flow construct including `if` , `while` , `foreach` , and `switch` statements creates a new scope boundary, and variables declared within these constructs are only accessible within that specific block and any nested blocks contained within it.

Variable lifetime is tied directly to scope, meaning that variables cease to exist when execution leaves the block in which they were declared. This automatic cleanup ensures predictable memory management and prevents accidental access to variables outside their intended scope.

Variable Shadowing

Variable shadowing (where variables declared in inner scopes can share the same identifier as variables in outer scopes) is permitted. When shadowing occurs, the inner variable takes precedence and the outer variable becomes temporarily inaccessible until execution exits the inner scope. This behavior is particularly relevant with `foreach` loops, where the iterator variable automatically shadows any existing variable that shares the same identifier, without generating warnings or errors.

Shadowing provides flexibility in variable naming while maintaining scope isolation, though developers should be aware that shadowed variables remain in memory and will become accessible again when the shadowing scope ends.

Data Context Restrictions

The identifier `Data` is reserved, and refers to the primary data object being processed by the script. Jyro prohibits certain operations on the `Data` identifier itself.

Attempting to declare `Data` as a variable using `var Data` syntax results in a runtime error. Similarly, direct assignment to `Data` such as `Data = newValue` is prohibited to prevent accidental replacement of the entire data context.

However, Jyro fully supports modification of the data context's child contents through property assignment and indexing operations. Scripts can freely modify data using syntax like `Data.property = value` or `Data[key] = value`, allowing comprehensive data transformation while maintaining the root context structure throughout script execution.

Important Considerations:

- Type annotations are optional but recommended for clarity
- Variables without initial values are implicitly null
- Variable names must follow identifier rules (letter followed by letters, digits, or underscores)

Assignment

Assignment statements modify the value of existing variables or properties within objects and arrays. The assignment target can include property access and indexing operations to modify nested data structures.

```
Assignment       = AssignmentTarget "=" Expression ;
AssignmentTarget = Identifier { ( "." Identifier | "[" Expression "]" ) } ;
```

Assignment operations evaluate the right-hand expression completely before storing the result in the specified target location. Chained property access and array indexing allow modification of deeply nested data structures in a single statement.

Valid Syntax:

```
counter = 5
user.name = "John"
items[0] = "first"
config["timeout"] = 30
nested.data[key] = value
```

Invalid Syntax:

```
5 = counter            # Cannot assign to literal
user.name.length = 10  # Cannot assign to method result
```

Important Considerations:

- Assignment targets support property access and indexing
- Chained property/index access is permitted
- Assignment is not an expression and cannot be used within other expressions

Expressions

Expression Statement

Expression statements evaluate standalone expressions primarily for their side effects, such as function calls that modify the data context or produce output. The result of the expression is computed but not stored unless the expression itself performs storage operations.

```
ExpressionStmt   = Expression ;
```

Valid Syntax:

```
CalculateTotal()
user.name
42
```

Expression Hierarchy

Expressions follow standard operator precedence rules organized from lowest to highest precedence. Each level of the hierarchy can contain multiple operators of the same precedence level, which are evaluated left-to-right.

Ternary Conditional Operator (`?:`)

The ternary conditional operator provides a concise way to select between two expressions based on a boolean condition. It follows the syntax `condition ? trueExpression : falseExpression` and evaluates to either the true expression or false expression depending on the truthiness of the condition.

```
Ternary          = LogicalOr [ "?" Ternary ":" Ternary ] ;
```

The ternary operator uses short-circuit evaluation, meaning only the condition and the selected expression are evaluated. If the condition is truthy, only the true expression executes; if falsy, only the false expression executes. This provides performance benefits and prevents side effects in the non-selected branch.

The ternary operator is right-associative, meaning nested ternary operators are grouped from right to left. Multiple ternary operators can be chained together, though readability may suffer with excessive nesting.

Valid Syntax:

```
status = isActive ? "active" : "inactive"
result = score >= 60 ? "pass" : "fail"
message = user.hasPermission ? GetWelcomeMessage() : "Access denied"
value = condition1 ? expr1 : condition2 ? expr2 : defaultExpr
```

Invalid Syntax:

```
isActive ? : "inactive"     # Missing true expression
isActive ? "active"         # Missing false expression and colon
isActive "active" : "inactive"  # Missing question mark
```

Important Considerations:

- The ternary operator has lower precedence than logical OR but higher precedence than assignment
- Right-associative: `a ? b : c ? d : e` is parsed as `a ? b : (c ? d : e)`
- Only the condition and selected expression are evaluated (short-circuit behavior)
- Both true and false expressions must be present

Logical OR (`or`)

The logical OR operator performs short-circuit evaluation, returning true if any operand evaluates to true. Evaluation stops at the first true operand, making it efficient for conditional chains.

```
LogicalOr        = LogicalAnd { "or" LogicalAnd } ;
```

Examples:

```
isValid or hasPermission
condition1 or condition2 or condition3
```

Logical AND (`and`)

The logical AND operator performs short-circuit evaluation, returning true only if all operands evaluate to true. Evaluation stops at the first false operand, preventing unnecessary computation.

```
LogicalAnd       = Equality { "and" Equality } ;
```

Examples:

```
isActive and hasAccess
x > 0 and x < 100
```

Equality (`==`, `!=`)

Equality operators compare values for equivalence or difference. The comparison behavior depends on the types of the operands and follows the runtime's equality semantics.

```
Equality         = Relational { ("==" | "!=") Relational } ;
```

Examples:

```
name == "admin"
status != "pending"
```

Relational (`<`, `<=`, `>`, `>=`, `is`)

Relational operators perform magnitude comparisons for numeric values and the `is` operator performs runtime type checking. Type checking returns true if the operand matches the specified type.

```
Relational       = Additive { ("<" | "<=" | ">" | ">=" | "is") Additive } ;
```

Examples:

```
age >= 18
score < 100
value is number
```

Additive (`+`, `-`)

Addition and subtraction operators perform arithmetic operations on numeric values. The behavior with non-numeric types depends on the runtime's type conversion and operation semantics.

```
Additive         = Multiplicative { ("+" | "-") Multiplicative } ;
```

Examples:

```
total + tax
endDate - startDate
```

Multiplicative (`*`, `/`, `%`)

Multiplication, division, and modulo operators perform arithmetic operations with higher precedence than additive operators. Division by zero behavior is defined by the runtime implementation.

```
Multiplicative   = Unary { ("*" | "/" | "%") Unary } ;
```

Examples:

```
quantity * price
total / count
value % 10
```

Unary (`not`, `-`)

Unary operators apply to single operands with the highest precedence among operators. The `not` operator performs logical negation, while the minus operator performs arithmetic negation.

```
Unary            = [ "not" | "-" ] Primary ;
```

Examples:

```
not isValid
-balance
not user.isActive
```

Important Considerations:

- Operator precedence follows mathematical conventions
- The `is` operator performs type checking
- Logical operators (`and` , `or` , `not`) use words, not symbols

- Chaining comparisons (e.g., `a < b < c`) is not supported

Primary Expressions

Primary expressions form the fundamental building blocks of more complex expressions, including literals, identifiers with property access, function calls, and parenthesized expressions.

```
Primary          = Literal
                 | Identifier { ( "." Identifier | "[" Expression "]" ) }
                 | FunctionCall
                 | "(" Expression ")"
                 | ObjectLiteral
                 | ArrayLiteral ;
```

Property access using dot notation and indexing using bracket notation can be chained to access deeply nested data structures. Function calls invoke host-provided functions with optional arguments. Parenthesized expressions override operator precedence by explicitly grouping sub-expressions.

Examples:

```
user.profile.name
items[index]
config["database"]["host"]
data.users[userId].permissions
(a + b) * c
not (isValid and hasPermission)
```

Literals and Data Structures

Basic Literals

Basic literals represent fundamental data values including numbers, strings, and boolean constants. The `null` literal represents the absence of a value.

```
Literal          = NumberLiteral
                 | StringLiteral
                 | "true"
                 | "false"
                 | "null" ;
```

Examples:

```
42
3.14159
"Hello, World!"
true
false
null
```

Object Literals

Object literals create key-value data structures. Keys can be string literals or computed expressions enclosed in brackets for dynamic key generation.

```
ObjectLiteral    = "{" [ ObjectEntry { "," ObjectEntry } ] "}" ;
ObjectEntry      = (StringLiteral | InterpolatedKey) ":" Expression ;
InterpolatedKey  = "[" Expression "]" ;
```

Interpolated keys are created where they don't exist.

Valid Syntax:

```
{}
{ "name": "John", "age": 30 }
{ "key1": value1, "key2": value2 }
{ [dynamicKey]: "value" }
{ "nested": { "inner": true } }
```

Invalid Syntax:

```
{ name: "John" }    # Keys must be quoted or interpolated
{ "key": }          # Missing value
```

Important Considerations:

- Keys must be string literals or computed using interpolated syntax `[expression]`
- Trailing commas are not permitted in object literals

Array Literals

Array literals create ordered collections. Elements are specified as a comma-separated list of expressions within brackets.

```
ArrayLiteral     = "[" [ Expression { "," Expression } ] "]" ;
```

Valid Syntax:

```
[]
[1, 2, 3]
["a", "b", "c"]
[{ "id": 1 }, { "id": 2 }]
```

Invalid Syntax:

```
[1, 2, 3,]          # Trailing comma not supported
```

Important Considerations:

- Trailing commas are not permitted in array literals
- Elements can be any valid expression including nested objects and arrays

Control Flow

Conditional Execution (`if`, `then`, `else`, `end`)

Conditional statements execute different code paths based on boolean expression evaluation. The `if` statement supports multiple `else if` branches for complex decision trees and an optional final `else` branch for default behavior.

```
IfStmt           = "if" Expression "then" { Statement }
                   { "else" "if" Expression "then" { Statement } }
                   [ "else" { Statement } ] "end" ;
```

Each condition is evaluated in sequence until one evaluates to true, at which point the corresponding statement block executes and control transfers to the statement following the `end` keyword. If no conditions evaluate to true, the optional `else` block executes.

Valid Syntax:

```
if condition then
    DoSomething()
end

if score >= 90 then
    grade = "A"
else if score >= 80 then
    grade = "B"
else
    grade = "F"
end

if condition then DoSomething() end  # Statements can be on the same line
```

Invalid Syntax:

```
if condition {              # Missing 'then'
    DoSomething()
}

if condition then           # Missing 'end'
    DoSomething()
```

Important Considerations:

- `if` statements must be terminated with `end`
- The `then` keyword is required after each condition
- `else if` is written as two separate keywords

Multi-way Branching (`switch`, `case`, `default`, `end`)

Switch statements provide multi-way branching based on expression value matching. Each `case` compares its expression against the switch expression for equality, executing the corresponding statement block when a match is found.

```
SwitchStmt       = "switch" Expression
                     { "case" Expression "then" { Statement } }
                     [ "default" { Statement } ]
                   "end" ;
```

Cases are evaluated in the order they appear until a matching value is found. The optional `default` case executes if no explicit cases match. Each case is independent with no fall-through behavior, eliminating the need for explicit `break` statements.

Valid Syntax:

```
switch status
case "pending" then
    ProcessPending()
case "approved" then
    ProcessApproved()
default
    HandleError()
end
```

Invalid Syntax:

```
switch status {             # Missing case statements and 'end'
case "pending":             # Missing 'then'
    ProcessPending()
}
```

Important Considerations:

- `switch` statements must be terminated with `end`
- Each `case` requires the `then` keyword
- `default` case does not use `then`
- No fall-through behavior; each case is independent
- Cases do not require `break` statements

Iteration and Looping

Conditional Iteration (`while`, `do`, `end`)

While loops execute a statement block repeatedly as long as the specified condition evaluates to true. The condition is evaluated before each iteration, meaning the loop body may not execute at all if the condition is initially false.

```
WhileStmt        = "while" Expression "do" { Statement } "end" ;
```

The loop continues until the condition evaluates to false or until a `break` statement is encountered within the loop body. Care must be taken to ensure the condition will eventually become false to avoid infinite loops.

Valid Syntax:

```
while counter < 10 do
    counter = counter + 1
    Process(counter)
end

while HasMoreData() do
    data = FetchNext()
    ProcessData(data)
end
```

Invalid Syntax:

```
while counter < 10 {        # Missing 'do'
    counter = counter + 1
}

while counter < 10 do       # Missing 'end'
    counter = counter + 1
```

Important Considerations:

- `while` loops must be terminated with `end`
- The `do` keyword is required after the condition
- Infinite loops are possible if the condition never becomes false

Collection Iteration (`foreach`, `in`, `do`, `end`)

The `foreach` statement executes a statement block for each element in a collection, automatically handling iteration over arrays and objects. An iterator variable is automatically declared and assigned each element value during iteration.

```
ForEachStmt      = "foreach" Identifier "in" Expression "do"
                     { Statement }
                   "end" ;
```

For arrays, the iterator variable receives each element value in order. For objects, the iterator variable receives each property value. The iterator variable is automatically scoped to the loop and shadows any existing variable with the same name.

Valid Syntax:

```
foreach item in Data.items do
    ProcessItem(item)
end

foreach user in userList do
    ValidateUser(user)
    SaveUser(user)
end

foreach item in array ["car", "box", "table", "dog"] do   # Note use of array to declare
an array literal
    ProcessItem(item)
end
```

Invalid Syntax:

```
foreach item of items do    # Must use 'in', not 'of'
    ProcessItem(item)
end

foreach items do            # Missing iterator variable and 'in'
    ProcessItem()
end
```

The `Expression` after `in` can be any valid expression, and `ArrayLiteral` is part of the `Primary` expressions. So yes, this should be syntactically valid:

The grammar supports any expression that evaluates to an iterable collection, so you these are also valid syntax:

```
foreach activeUser in GetActiveUsers() do      # Assuming host-side function
GetActiveUsers() returns an array

foreach entry in (condition and arrayA or arrayB) do       # All conditions must result in
an array
```

Important Considerations:

- The iterator variable is automatically declared and scoped to the loop
- Use `in` keyword, not `of` or other variants
- The expression must evaluate to an iterable collection
- Iterator variable shadows any existing variable with the same name

Loop Control (`break`, `continue`)

Loop control statements modify the normal flow of iteration within loops. The `break` statement immediately exits the innermost containing loop, while `continue` skips the remaining statements in the current iteration and proceeds to the next iteration. All loop blocks must be terminated by an `end` statement.

```
BreakStmt        = "break" ;
ContinueStmt     = "continue" ;
```

These statements provide fine-grained control over loop execution, allowing early termination or selective processing of loop iterations based on runtime conditions.

Valid Syntax:

```
while true do
    if shouldExit then
        break
    end
    if shouldSkip then
        continue
    end
    ProcessItem()
end

foreach item in items do
    if item.isInvalid then
        continue
    end
    ProcessValidItem(item)
end
```

Important Considerations:

- `break` and `continue` can only be used within loop constructs
- `break` exits the innermost containing loop
- `continue` skips to the next iteration of the innermost containing loop

Function Integration

Function Calls

Function calls invoke host-provided functions with optional arguments passed as expressions. Functions extend the language capabilities by providing access to external data sources, computational operations, and system services that are not built into the core language.

```
FunctionCall     = Identifier "(" [ Expression { "," Expression } ] ")" ;
```

Arguments are evaluated from left to right before being passed to the function. The function identifier must match a function provided by the host environment, as the language does not support user-defined functions within scripts.

Jyro comes with a standard library that provides common functionality. Hosts can extend the function set available to Jyro by registering host-side functions with the Jyro runtime. These functions are then available to call within Jyro scripts, providing unlimited host interop. The casing convention for host-side function registration (and the casing convention used by the standard library) is PascalCase.

> ⚠️ Hosts must ensure that the functions provided to Jyro scripts cannot result in privilege escalation or other security issues, especially when running untrusted scripts.

Valid Syntax:

```
GetData()
CalculateSum(a, b)
ProcessUser(user.id, user.name, user.isActive)
FormatMessage("Hello", name, GetCurrentTime())
```

Invalid Syntax:

```
GetData                     # Missing parentheses
CalculateSum(a, b,)        # Trailing comma not permitted
ProcessUser(,name)         # Missing argument
```

Important Considerations:

- Function calls always require parentheses, even with no arguments
- Arguments are evaluated left to right
- Trailing commas in argument lists are not permitted
- Functions must be provided by the host environment

Script Termination

Return Statement (`return`)

The `return` statement immediately halts script execution and makes the current `Data` context available to the caller. As the host and script share a single mutable `Data` context, Jyro's `return` statement does not carry an expression value.

```
ReturnStmt       = "return" ;
```

When `return` is encountered, all remaining statements in the script are skipped and control returns to the host environment or calling context (i.e. the calling script). The data object reflects all modifications made up to the point where `return` was executed. Scripts should therefore ensure that `Data` is in a valid state before invoking `return` (e.g. recovery from an error condition).

Valid Syntax:

```
return
```

Important Considerations:

- `return` is always written without an expression
- Execution stops immediately when `return` is encountered
- The current `Data` context is returned to the caller

Type Operations

Type Checking (`is`)

The `is` operator performs runtime type checking, returning true if the left operand matches the specified type. This enables scripts to make decisions based on the actual types of values encountered during execution.

```
Relational       = Additive { ("<" | "<=" | ">" | ">=" | "is") Additive } ;
```

Type checking occurs at runtime and can be used in conditional statements to handle different data types appropriately within the same script execution path.

Valid Syntax:

```
if value is number then
    PerformNumericOperation(value)
end

if data is object then
    ProcessObjectData(data)
end
```

Supported Type Checks:

- `value is number`
- `value is string`
- `value is boolean`
- `value is object`
- `value is array`

Type Checking Cascade Rules:

The `is` operator checks the runtime type of the evaluated expression, not the literal syntax. The cascade follows this hierarchy:

1. **`null`** - Null values return `false` for all type checks
2. **`number`** - Numeric values (integers and decimals), regardless of how they were created
3. **`string`** - Text values enclosed in quotes or returned from string operations
4. **`boolean`** - The literals `true` and `false`, or results of boolean operations
5. **`array`** - Collections created with `array []` syntax or returned from functions
6. **`object`** - Key-value structures created with `object {}` syntax or returned from functions

Examples:

```
42 is number          # true (number literal)
"42" is string        # true (string literal)
"42" is number        # false (string containing digits)
ToNumber("42") is number  # true (converted to number)
array [1,2,3] is array    # true
object {} is object       # true
null is number            # false
null is string            # false
```

Scripts can change variable types at runtime by invoking standard library conversion functions (e.g., `ToNumber("42")` to convert string "42" to number 42).

Important Considerations:

- Type checking is performed at runtime
- The `is` operator returns a boolean value
- Null values will not match any specific type

Lexical Conventions

Identifiers

Identifiers name variables, functions, and properties within the language. They must begin with a letter and can contain letters, digits, and underscores in any combination.

```
Identifier       = Letter { Letter | Digit | "_" } ;
Letter           = "A"…"Z" | "a"…"z" ;
Digit            = "0"…"9" ;
```

Important Considerations:

- Must start with a letter (A-Z, a-z)
- Can contain letters, digits, and underscores
- Case-sensitive

Number Literals

Number literals represent numeric values in both integer and decimal formats. Scientific notation is not supported.

```
NumberLiteral    = Digit { Digit } [ "." Digit { Digit } ] ;
```

Important Considerations:

- Integer and decimal numbers supported
- No scientific notation
- No leading zeros (except for `0` itself)

String Literals

String literals represent text values enclosed in double quotation marks.

```
StringLiteral    = '"' { Character } '"' ;
Character        = ? any character except " or newline ? ;
```

Important Considerations:

- Enclosed in double quotes
- Cannot contain newlines

Reserved Keywords

The following keywords are reserved by the language and cannot be used as identifiers:

The Data Context: `Data`

Variable Declaration: `var`

Type Names: `number` , `string` , `boolean` , `object` , `array`

Control Flow: `if` , `then` , `else` , `end` , `switch` , `case` , `default` , `return`

Iteration: `foreach` , `in` , `do` , `while` , `break` , `continue`

Logical Operators: `and` , `or` , `not`

Type Checking: `is`

Literals: `true` , `false` , `null`

Operator Precedence Summary

Operators are listed from highest to lowest precedence:

1. Primary expressions (literals, identifiers, function calls, parentheses)
2. Unary operators (`not` , `-`)
3. Multiplicative (`*` , `/` , `%`)
4. Additive (`+` , `-`)
5. Relational (`<` , `<=` , `>` , `>=` , `is`)
6. Equality (`==` , `!=`)
7. Logical AND (`and`)
8. Logical OR (`or`)
9. Ternary conditional (`?:`)